Iron Dome Game - Homework Assignment

Last edited: January 28, 2026

Overview

You are required to implement a console-based **2D Iron Dome** game simulation. The game contains two main systems:

- **Rocket launcher (threat generator):** periodically launches airborne **plates** (targets) that move across the grid.
- **Cannon (defense):** fires **interceptor rockets** to shoot down plates before they escape the play area.

The player fires an interceptor by pressing the **Enter** key. The simulation ends when a game-over condition is met (e.g., time limit, number of escaped plates, or another rule defined in the template). At the end of the run, the program prints a **game statistics** summary that reports all relevant performance metrics.

Please review the provided template code and implement the missing parts. You may refactor and improve the code if you find design issues, poor coding practices, or opportunities to add small features that improve usability and robustness.

Evaluation focus

- Proper use of data structures (e.g., **std::vector**, **std::stringstream**)
- Correct statistical calculations
- Efficient algorithms
- Clean API design
- Edge-case handling

Implementation Guidelines

Code quality requirements

Object-oriented design

- Use inheritance and polymorphism appropriately.
- Build a clean, minimal class hierarchy with well-defined responsibilities.
- Use virtual functions where dynamic dispatch is required; avoid it where it is not.

Code organization

- Keep a clear separation of concerns (rendering, physics, input, statistics, etc.).
- Maintain a logical file structure and consistent naming conventions.

Error handling

- Handle null pointers safely (or avoid raw owning pointers altogether).
- Check bounds before array or grid access.
- Validate input parameters and handle invalid states gracefully.

C++ best practices

- Use smart pointers (e.g., **std::shared_ptr**) where ownership is shared; prefer **std::unique_ptr** when ownership is exclusive.
- Prefer const-correctness for parameters and member functions.
- Use meaningful names and keep functions small and focused.
- Comment only where the logic is non-obvious; avoid redundant comments.

Performance

- Use efficient algorithms and avoid unnecessary recalculations in the main loop.
- Remove expired or inactive entities to keep per-tick work bounded.

Building and Running

Build command

Using the provided script:

```
./build.sh
```

Or compile manually:

```
g++ grid.cpp entity.cpp plate.cpp game.cpp pitcher.cpp \
    game_statistics.cpp cannon.cpp rocket.cpp main.cpp \
    -I$(pwd)/include -lpthread -o iron_dome_game
```

Run

```
./iron_dome_game
```

Controls

- Press **Enter** to fire an interceptor rocket from the cannon.
- (Optional) If you add extra controls, document them clearly in the README.

Submission Requirements

1. Source code

- All **.cpp** and **.hpp** files.
- Update **build.sh** if you added new files.

- Code must compile without errors on a Linux environment.

2. Documentation

- A brief **README** explaining your implementation choices.
- Comments in code for complex logic and non-obvious decisions.
- Notes about design decisions, assumptions, and any optional features.

3. Testing

- Game runs without crashes (including edge cases).
- All core features work as expected.
- Optional features are clearly documented (how to use them and how they affect stats).

Notes

- The initial project compiles and runs; start by analyzing the existing code structure.
- You may modify existing files as needed, but maintain the overall architecture unless you have a clear reason to improve it.
- Prioritize code quality over feature quantity.
- External libraries are allowed but not required.
- If anything is unclear, state your assumptions in the README.